

Data260 - HW1

Rachandeep Kaur

Github Link- <https://github.com/rachandeepkaur/data260-7117.git>

Deliverables

Part 1

HTML code for the form creation

```
code > web_application > HW1-RachandeepKaur.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <!-- Page title with HW1 - {Name} -->
7   <title>HW1 - Rachandeep Kaur</title>
8   <style>--
83 </style>
84 </head>
85 <body>
86
87   <!-- Heading naming assigned domain entity using <h1> (biggest font size) -->
88   <h1>Local Restaurant Inspections</h1>
89
90   <!-- Form for domain entity -->
91   <form id="inspectionForm">--
172 </form>
173
174   <!-- Shown while script.js is waiting on the FastAPI /api/analyze response -->
175   <!-- This wasn't mentioned in the requirements but I just added so that anyone checking the application wait for agent call -->
176   <div id="loader" class="loader" hidden></div>
177
178   <!-- Populated by script.js with the FastAPI /api/analyze response -->
179   <div id="analysisResults"></div>
180
181 </body>
182 <footer>
183   <!-- Script tag linking JavaScript file at the end of the HTML file -->
184   <script src="script.js"></script>
185
186 </footer>
187 </html>
```

code > web_application > HW1-RachandeepKaur.html > html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <!-- Page title with HW1 - {Name} -->
7    <title>HW1 - Rachandeep Kaur</title>
8  </head>
83 </style>
84 </head>
85 <body>
86
87 <!-- Heading naming assigned domain entity using <h1> (biggest font size) -->
88 <h1>Local Restaurant Inspections</h1>
89
90 <!-- Form for domain entity -->
91 <form id="inspectionForm">
92
93   <!-- Requirement I.4.a: Required text input for primary field with autofocus & placeholder -->
94   <div>
95     <label for="facility_name">Facility Name / ID:</label><br>
96     <input
97       type="text"
98       id="facility_name"
99       name="facility_name"
100      placeholder="FA0206933 - YUMMY KITCHEN"
101      autofocus
102      required
103     />
104   </div>
105   <br>
106
107   <!-- Requirement I.4.b: Required text input for secondary field with descriptive placeholder -->
108   <div>
109     <label for="site_address">Program Site Address:</label><br>
110     <input
111       type="text"
112       id="site_address"
113       name="site_address"
114       placeholder="1711 BRANHAM LN A9, SAN JOSE, CA 95118"
115       required
116     />
117   </div>
118   <br>
119
120   <!-- Requirement I.4.c: Required email input with descriptive placeholder -->
121   <div>
122     <label for="inspector_email">Submitter Email:</label><br>
123     <input
124       type="email"
125       id="inspector_email"
126       name="inspector_email"
127       placeholder="inspector@countyhealth.gov"
128       required
129     />
130   </div>

```

```
ms README.md U HW1-RachandeepKaur.html U X agents_demo.py U
code > web_application > HW1-RachandeepKaur.html > html
  2  <html lang="en">
  85  <body>
  91  <form id="inspectionForm">
134    <div>
136      <textarea
138        name="inspection_summary"
139        rows="5"
140        cols="50"
141        placeholder="Provide inspection observations and compliance details (must be > 25 characters)..."
142        required
143      ></textarea>
144    </div>
145    <br>
146
147    <!-- Requirement I.4.e: Dropdown with 4 domain-appropriate options -->
148    <div>
149      <label form="program_element">Program Element / Category:</label><br>
150      <select id="program_element" name="program_element" required>
151        <option value="" disabled selected>-- Select Inspection Category --</option>
152        <option value="FOOD PREP / FOOD SVC OP 0-5 EMPLOYEES RC 3 - FP11">FOOD PREP / FOOD SVC OP 0-5 EMPLOYEES RC 3 - FP11</option>
153        <option value="FOOD PREP / FOOD SVC OP 6-25 EMPLOYEES">FOOD PREP / FOOD SVC OP 6-25 EMPLOYEES</option>
154        <option value="ROUTINE HEALTH INSPECTION">ROUTINE HEALTH INSPECTION</option>
155        <option value="FOLLOW-UP COMPLIANCE CHECK">FOLLOW-UP COMPLIANCE CHECK</option>
156      </select>
157    </div>
158    <br>
159
160    <!-- Requirement I.4.f: Checkbox and exact label text -->
161    <div class="checkbox-row">
162      <input type="checkbox" id="terms" name="terms" required />
163      <label for="terms">I agree to the terms and conditions.</label>
164    </div>
165    <br>
166
167    <!-- Requirement I.4.g: Submit button with domain-appropriate text -->
168    <div>
169      <button type="submit">Submit Inspection Report</button>
170    </div>
171
172  </form>
173
174  <!-- Shown while script.js is waiting on the FastAPI /api/analyze response -->
175  <!-- This wasn't mentioned in the requirements but I just added so that anyone checking the application wait for agent call -->
176  <div id="loader" class="loader" hidden></div>
177
178  <!-- Populated by script.js with the FastAPI /api/analyze response -->
179  <div id="analysisResults"></div>
180
181 </body>
182 <footer>
183   <!-- Script tag linking JavaScript file at the end of the HTML file -->
184   <script src="script.js"></script>
185
186 </footer>
187 </html>
```

Javascript file for form validation and submitting the input to FastApi

```
code > web_application > JS script.js > document.addEventListener("DOMContentLoaded") callback > validateForm
41
42 document.addEventListener("DOMContentLoaded", () => {
43     const form = document.getElementById("inspectionForm");
44
45     // Validation using an Arrow Function
46     const validateForm = (summary, isChecked) => {
47         // Verify content field has strictly more than 25 characters
48         if (summary.trim().length <= 25) {
49             alert("Validation Error: Inspection Findings Summary must be more than 25 characters long.");
50             return false;
51         }
52
53         // Verify terms and conditions checkbox is checked
54         if (!isChecked) {
55             alert("Validation Error: You must agree to the terms and conditions before submitting.");
56             return false;
57         }
58
59         return true;
60     };
61
62     form.addEventListener("submit", async (event) => {
63         event.preventDefault();
64
65         // Get input field values
66         const facilityName = document.getElementById("facility_name").value;
67         const siteAddress = document.getElementById("site_address").value;
68         const inspectorEmail = document.getElementById("inspector_email").value;
69         const inspectionSummary = document.getElementById("inspection_summary").value;
70         const programElement = document.getElementById("program_element").value;
71         const termsChecked = document.getElementById("terms").checked;
72
73         // Run Arrow Function Validation - stop execution if invalid
74         if (!validateForm(inspectionSummary, termsChecked)) {
75             return;
76         }
77
78         // Convert form data into a JSON string and log output
79         const formDataObj = {
80             facility_name: facilityName,
81             site_address: siteAddress,
82             inspector_email: inspectorEmail,
83             inspection_summary: inspectionSummary,
84             program_element: programElement,
85             agreed_to_terms: termsChecked
86         };
87
88         const jsonString = JSON.stringify(formDataObj);
89         console.log("Form Data (JSON String):", jsonString);
90
91         //Object destructuring to extract primary field and email field
92         const parsedData = JSON.parse(jsonString);
93         const { facility_name, inspector_email } = parsedData;
94         console.log("Destructured Facility Name:", facility_name);
95         console.log("Destructured Inspector Email:", inspector_email);
96     });
97 }
```

```

const jsonString = JSON.stringify(formDataObj);
console.log("Form Data (JSON String):", jsonString);

//Object destructuring to extract primary field and email field
const parsedData = JSON.parse(jsonString);
const { facility_name, inspector_email } = parsedData;
console.log("Destructured Facility Name:", facility_name);
console.log("Destructured Inspector Email:", inspector_email);

// Spread operator to add submissionDate
const updatedParsedData = {
  ...parsedData,
  submissionDate: new Date().toISOString()
};
console.log("Updated Parsed Object with Submission Date:", updatedParsedData);

// IIncrement and log count via closure only upon successful submission
const currentCount = getSubmissionCount();
// const currentCount = countInc++;
console.log(`Successful Submission Count: ${currentCount}`);

alert(`Form submitted successfully! Total successful submissions: ${currentCount}`);

// Send the submission to the FastAPI backend
const loader = document.getElementById("loader");
loader.hidden = false;

try {
  const response = await fetch("/api/analyze", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: jsonString
  });

  if (!response.ok) {
    throw new Error(`API responded with status ${response.status}`);
  }

  const publish = await response.json();
  console.log("Agent Pipeline Result:", publish);
  renderAnalysis(publish);
} catch (error) {
  console.error("Agent analysis request failed:", error);
  alert("Form submitted, but the agent analysis request failed. See console for details.");
} finally {
  loader.hidden = true;
}
});
});

```

```
code > web_application > JS script.js > ...
1 // Requirement Closure to track successful form submissions
2 // var countInc = 0;
3 const createSubmissionCounter = () => {
4   let count = 0;
5   return () => {
6     count++;
7     console.log(`Submission count (Closure): ${count}`);
8     return count;
9   };
10 };
11
12 const getSubmissionCount = createSubmissionCounter();
13
14 // Renders the JSON returned by the FastAPI
15 > const renderAnalysis = (publish) => { ...
40 };
41
42 document.addEventListener("DOMContentLoaded", () => {
43   const form = document.getElementById("inspectionForm");
44
45   // Validation using an Arrow Function
46   const validateForm = (summary, isChecked) => {
47     // II.1.a: Verify content field has strictly more than 25 characters
48     if (summary.trim().length <= 25) {
49       alert("Validation Error: Inspection Findings Summary must be more than 25 characters long.");
50       return false;
51     }
52
53     // Verify terms and conditions checkbox is checked
54     if (!isChecked) {
55       alert("Validation Error: You must agree to the terms and conditions before submitting.");
56       return false;
57     }
58
59     return true;
60   };
61
62 > form.addEventListener("submit", async (event) => { ...
135 });
136 });
```

Screenshot of running through docker

localhost:8817

Local Restaurant Inspections

Facility Name / ID:

FA0206933 - YUMMY KITCHEN

Program Site Address:

1711 BRANHAM LN A9, SAN JOSE, CA 95118

Submitter Email:

inspector@countyhealth.gov

Inspection Findings Summary:

Provide inspection observations and compliance details (must be > 25 characters)...

Program Element / Category:

-- Select Inspection Category --

☐ I agree to the terms and conditions.

Submit Inspection Report

Steps followed for creating docker image and pushing it to AWS ECR

Step1: write the dockerfile in code folder

```
Dockerfile U X agents_demo.py U model_client.py U hw1_client.py
code > Dockerfile > ...
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY code ./code
9
10 WORKDIR /app/code/web_application
11
12 ENV OLLAMA_HOST=http://host.docker.internal:11434
13
14 EXPOSE 8817
15
16 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8817"]
17
```

Step 2: run the command in the terminal to create the Docker image

```
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % docker build -f code/Dockerfile -t data260-7117 .
[+] Building 1.4s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 335B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/6] FROM docker.io/library/python:3.12-slim@sha256:09f7da3bc104798d0afb40bc08d23ab2da20a76130cec1f2ef170848f5d85217
=> => resolve docker.io/library/python:3.12-slim@sha256:09f7da3bc104798d0afb40bc08d23ab2da20a76130cec1f2ef170848f5d85217
=> [internal] load build context
=> => transferring context: 1.12kB
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY requirements.txt .
=> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/6] COPY code ./code
=> CACHED [6/6] WORKDIR /app/code/web_application
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:7c56bb8654fecc524c6e6933651aed40d1683d92f8e8674af5e71fa57c912422
=> => exporting config sha256:c1a9e35ca64487260463d6126d6310a1d5475e77f7fcec2424b649d6c3d02e9
=> => exporting attestation manifest sha256:2b972455da21e8a90a1aec2714bd5a0cb870aeff1cbc2a4f19858351b65cfdcd
=> => exporting manifest list sha256:0c85583f0a6bdf7b612f9eb1e74f59cc93fd09f4b04c4641f7076a4c6586dc06
=> => naming to docker.io/library/data260-7117:latest
=> => unpacking to docker.io/library/data260-7117:latest

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/n0h7m5n4uxqoy1g0bgkhn29ci
```

Step 3: Login to aws through terminal and push the build

```

(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % aws ecr get-login-password --region us-east-2 | docker login --username AWS --password-stdin 080665850023.dkr.ecr.us-east-2.amazonaws.com
Login Succeeded
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % docker push 080665850023.dkr.ecr.us-east-2.amazonaws.com/data260-7117/my-web-app:latest
The push refers to repository [080665850023.dkr.ecr.us-east-2.amazonaws.com/data260-7117/my-web-app]
b79f58b3eb94: Pushed
69a569529bda: Pushed
6310eb16bf42: Pushed
9925f96b8824: Pushed
4f4fb700ef54: Layer already exists
51be1ac9a36b: Pushed
a0d8783128af: Pushed
43310b5ca065: Pushed
48579d87405c: Pushed
f420f26b543b: Pushed
latest: digest: sha256:0c85583f0a6bdf7b612f9eb1e74f59cc93fd09f4b04c4641f7076a4c6586dc06 size: 856
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 %

```

Step 4: Deploy in ECS

The screenshot displays the AWS Management Console for the Amazon Elastic Container Service (ECS). The breadcrumb navigation shows the path: Amazon Elastic Container Service > Clusters > my-web-app-cluster > Services. The left-hand navigation pane includes links for Favourites and recents, Express Mode, Clusters, Namespaces, Task definitions, Daemon task definitions, Account settings, AWS Resource Explorer, AWS Batch, Amazon ECR, Repositories, Online learning workshop, Documentation, Discover products, and Subscriptions.

The main content area is titled 'my-web-app-cluster' and shows it was last updated on 31 August 2026, 12:58 (UTC-7:00). It includes a 'Cluster overview' section with the following details:

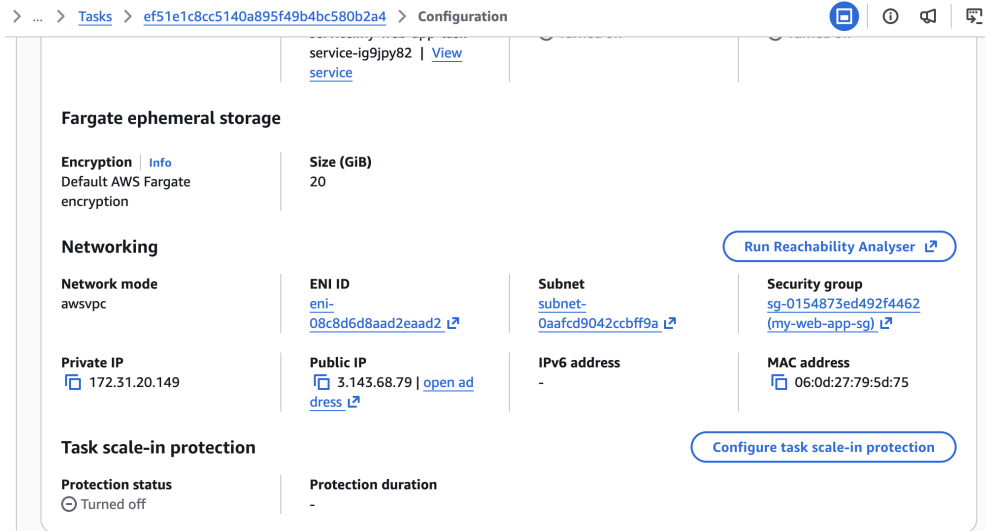
- Cluster ARN:** arn:aws:ecs:us-east-2:0...
- Status:** Active
- CloudWatch monitoring:** Default
- Action logs:** Turned off
- Services:** 1 active | 0 draining
- Tasks:** 1 running | 0 pending
- Registered container instances:** -

Below the overview, there are tabs for Services, Tasks, Daemons, Infrastructure, Metrics, Scheduled tasks, and Configuration - update. The 'Services' tab is selected, showing 'Services (1)' with an 'Info' icon. It includes buttons for 'Manage tags', 'Update', 'Delete service', and 'Create'. There are also filters for launch type, scheduling strategy, and resource management type.

A table lists the services:

☐	Service name	ARN	Status	Schedu...	L...	Task de...
☐	my-web-app-task-service-ig9jpy82	arn:aws:ec...	Active	Replica	Far...	my-web-a...

Step 5: Create Security group



Provide a screenshot of your app running on your public IP address.

← → ↺ ⚠ Not Secure 3.143.68.79/HW01-RachandeepKaur.html

Local Restaurant Inspections

Facility Name / ID:

FA0206933 - YUMMY K

Program Site Address:

1711 BRANHAM LN A9

Submitter Email:

inspector@countyhealthf

Inspection Findings Summary:

Provide inspection observations and compliance details (must be > 25 characters)...

Program Element / Category:

-- Select Inspection Category --

☐ I agree to the terms and conditions.

Submit Inspection Report

Part 2

1. Command used (the exact python agents_demo.py line).

python code/agents_demo.py

```
def run_planner(client: ModelClient, title: str, content: str) -> PlannerOutput:
    user = f'Title: "{title}"\n\nContent:\n{content}'
    return client.call(system=PLANNER_SYSTEM, user=user, output_format=PlannerOutput)

def run_reviewer(client: ModelClient, title: str, content: str, draft: PlannerOutput) -> ReviewerOutput:
    user = (
        f'Title: "{title}"\n\nContent:\n{content}\n\n'
        f'Planner's draft (JSON):\n{draft.model_dump_json()}'
    )
    return client.call(system=REVIEWER_SYSTEM, user=user, output_format=ReviewerOutput)

def finalize(title: str, review: ReviewerOutput) -> PublishOutput:
    """ finalization step - no model call, just assembles the Publish record."""
    return PublishOutput(
        title=title,
        tags=review.revised_output.tags,
        summary=review.revised_output.summary,
        approved_by_reviewer=review.approved,
    )

def build_document(submission: InspectionSubmission) -> Tuple[str, str]:
    """Turn a form submission into the title/content pair the agents read."""
    title = f"{submission.program_element.title()} - {submission.facility_name}"
    content = (
        f'Facility: {submission.facility_name}\n'
        f'Site Address: {submission.site_address}\n'
        f'Program Element: {submission.program_element}\n'
        f'Inspector: {submission.inspector_email}\n'
        f'Findings: {submission.inspection_summary}'
    )
    return title, content

def run_pipeline(
    client: ModelClient, submission: InspectionSubmission
) -> Tuple[PlannerOutput, ReviewerOutput, PublishOutput]:
    """Run Planner -> Reviewer -> finalize for one submission.

    Raises 'ModelClientError' if the local model is unreachable or never
    returns valid JSON - callers (CLI, FastAPI) decide how to surface that.
    """
    title, content = build_document(submission)
    draft = run_planner(client, title, content)
    review = run_reviewer(client, title, content, draft)
    publish = finalize(title, review)
    return draft, review, publish
```

```

(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % python code/agents_demo.py

=====
PLANNER
=====
{
  "tags": [
    "health inspection",
    "food safety",
    "facility compliance"
  ],
  "draft_summary": "YUMMY KITCHEN faced issues with cold-holding temperatures and missing signage, requiring corrective actions within 48 hours."
}

=====
REVIEWER
=====
{
  "approved": true,
  "feedback": "",
  "revised_output": {
    "tags": [
      "health inspection",
      "food safety",
      "facility compliance"
    ],
    "summary": "YUMMY KITCHEN had cold-holding issues and missing signage, requiring 48-hour corrective actions."
  }
}

=====
PUBLISH
=====
{
  "title": "Routine Health Inspection - FA0206933 - YUMMY KITCHEN",
  "tags": [
    "health inspection",
    "food safety",
    "facility compliance"
  ],
  "summary": "YUMMY KITCHEN had cold-holding issues and missing signage, requiring 48-hour corrective actions.",
  "approved_by_reviewer": true
}

```

2. Console screenshot(s) showing: Planner output, Reviewer output, Finalized output.

```

(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % python code/agents_demo.py

=====
PLANNER (41221 ms)
=====
{
  "tags": [
    "health inspection",
    "food safety",
    "facility compliance"
  ],
  "draft_summary": "YUMMY KITCHEN faced issues with cold-holding temperatures and missing signage, requiring corrective actions within 48 hours."
}

=====
REVIEWER (32333 ms)
=====
{
  "approved": true,
  "feedback": "",
  "revised_output": {
    "tags": [
      "health inspection",
      "food safety",
      "facility compliance"
    ],
    "summary": "YUMMY KITCHEN had cold-holding issues and missing signage, requiring 48-hour corrective actions."
  }
}

=====
PUBLISH
=====
{
  "title": "Routine Health Inspection - FA0206933 - YUMMY KITCHEN",
  "tags": [
    "health inspection",
    "food safety",
    "facility compliance"
  ],
  "summary": "YUMMY KITCHEN had cold-holding issues and missing signage, requiring 48-hour corrective actions.",
  "approved_by_reviewer": true
}

```

3. Short answers (1–2 lines each):

Q1. The 3 final tags you obtained.

Final tags: health inspection, food safety, facility compliance

Q2. The final summary(≤25 words).

"YUMMY KITCHEN had cold-holding issues and missing signage, requiring 48-hour corrective actions."

Q3. Did the Reviewer agent change anything? (yes/no + explanation)

Yes, it marked the summary approved but still changed it slightly and make it more concise, changing the wording of the sentence.

4. An explanation of each step in your own words.

For the very first step of this part, I go through the documentation of Langchain and read about the “Create an agent”. By referencing the code snippet in the documentation, I created one simple planner agent and passed the sample output to test if it's working. This includes importing the ChatOllama from langchain_ollama, ChatPromptTemplate, MessagesPlaceholder from langchain_core.prompts. And other required modules to complete the agent code which includes the llm call(ollama:qwen3-8b). After completing the initial code I run the agent with command “python code/agents_demo.py” and after getting successful output I wrote the code for second (“reviewer”) agent and then finalise the output which includes just to wrap the output without calling any LLM. While performing these steps I make sure to print all the outputs. When the basic functionality works, I also included pydantic for typed data shape instead of passing raw dict and it also helps in validating the LLM’s output using output_format.model_validate(data). I improve the system prompts to match the requirements mentioned like what it specifically need to extract and what is output format.

After completing the agent calls with static sample data I integrated it with dynamic user inputs from javascript file. This includes writing the api using FastAPI and defining the routes and adding middleware. And then integrating this api on form submission to get user inputs. And passing this inputs to build_document in agents_demo file to replace the static sample inputs with domain specific inputs. And again testing the whole process.

Part 3

Run your agents_demo.py pipeline on that exact input 20 times at temperature 0.7, and 20 times at temperature 0.0 (40 runs total).

M+ METRICS.md U X

reports > hw01 > METRICS.md

Preview Markdown

HW01 Metrics — Planner/Reviewer Non-Determinism Study

Fixed input (reports/hw01/cases/nondeterminism_input.json , title: "Routine Health Inspection - FA0206933 - YUMMY KITCHEN"), run through the real Planner -> Reviewer -> Publish pipeline (qwen3:8b via Ollama) 20 times at each of temperature=0.7 and temperature=0.0 (40 runs total). Source data: {} raw/nondeterminism_runs.json, raw/nondeterminism_runs.csv. Full console transcript: RUN_LOG.txt.

Temperature = 0.7

- Runs completed: 20/20 (0 failed)
- Distinct tag sets produced: 3
- Tags that appeared in all 20 runs: health inspection
- Tags that appeared in exactly one run: handwashing signage

Latency (n=19)	p50	p95	p99
ms	24656.0	28790.1	29374.0

Excluded from the latency stats above: run 19 (23,302,721 ms) — latency this many multiples (5x+) above the group median indicates the host machine slept mid-call (wall-clock time includes sleep time), not real model latency. Still included in the tag-set analysis above and present in the raw data.

Temperature = 0.0

- Runs completed: 20/20 (0 failed)
- Distinct tag sets produced: 1
- Tags that appeared in all 20 runs: facility compliance, food safety, health inspection
- Tags that appeared in exactly one run: (none)

Latency (n=20)	p50	p95	p99
ms	23167.5	25365.1	25913.8

Save the raw per-run results (tags + latency, all 40 runs) as JSON/CSV in reports/hw01/raw/

reports/hw01

cases

{} nondeterminism_input.json U

raw

raw/nondeterminism_runs.csv U

raw/nondeterminism_runs.json U

AI_USE.md U

METRICS.md U

README.md U

report.pdf U

RUN_LOG.txt U

{} verification.json U

```

reports > hw01 > raw > nondeterminism_runs.csv > data
1 temperature,run,tags,summary,approved,latency_ms,planner_ms,reviewer_ms,planner_total_tokens,reviewer_total_tokens,started_at,finished_at,status,error
2 0.7,1,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding issues and missing handwashing signs. Corrections made on-site; signage to be
3 0.7,2,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding issues and missing handwashing signs, requiring corrective actions within 48
4 0.7,3,health inspection; food safety; facility compliance,"Routine health inspection found cold-holding issues and missing signage at YUMMY KITCHEN, with corrections
5 0.7,4,health inspection; food safety; cold-holding temperatures,"YUMMY KITCHEN had improper cold-holding temperatures and missing handwashing signs, corrected on-site
6 0.7,5,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding issues and missing signage, requiring corrections within 48 hours.",True,247
7 0.7,6,health Col 3: tags food safety; cold-holding temperatures,YUMMY KITCHEN had cold-holding temperature issues and missing signage during a health inspection. Manag
8 0.7,7,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding issues and missing signage, needing corrections within 48 hours.",True,25106
9 0.7,8,health inspection; food safety; cold-holding temperatures,"YUMMY KITCHEN had cold-holding temperature issues and missing signage during a health inspection, req
10 0.7,9,health inspection; food safety; facility compliance,Yummy Kitchen had cold-holding temperature issues and missing handwashing signs during a health inspection.,
11 0.7,10,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
12 0.7,11,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
13 0.7,12,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding issues and missing signage, requiring 48-hour corrective actions.",True,232
14 0.7,13,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding issues and missing signage, requiring corrections within 48 hours.",True,23
15 0.7,14,health inspection; cold-holding temperatures; handwashing signage,YUMMY KITCHEN had cold-holding temperature and handwashing signage issues during a health ins
16 0.7,15,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
17 0.7,16,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs, requiring 48-hour correct
18 0.7,17,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
19 0.7,18,health inspection; food safety; facility compliance,Yummy Kitchen had cold-holding temperature issues and missing handwashing signs during a health inspection.,
20 0.7,19,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding temperature issues and missing signage, requiring 48-hour corrective action
21 0.7,20,health inspection; food safety; facility compliance,"YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs, requiring corrective acti
22 0.0,1,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
23 0.0,2,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
24 0.0,3,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
25 0.0,4,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
26 0.0,5,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
27 0.0,6,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
28 0.0,7,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
29 0.0,8,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
30 0.0,9,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
31 0.0,10,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
32 0.0,11,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
33 0.0,12,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
34 0.0,13,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
35 0.0,14,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
36 0.0,15,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
37 0.0,16,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
38 0.0,17,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
39 0.0,18,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
40 0.0,19,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
41 0.0,20,health inspection; food safety; facility compliance,YUMMY KITCHEN had cold-holding temperature issues and missing handwashing signs during a health inspection.,
42

```

Using your own numbers, explain what two users sending identical input might see, and give one example where run-to-run variation is acceptable and one where it is not.

When two users send identical input, it depends on the temperature. As data for 40 non-deterministic shows that at temperature=0.0, all 20 identical runs produced the same 3 tags and almost same wording, but when increased temperature=0.7, which means it grows the model creativity and it starts producing somewhat different results for each identical run. As the agent_model has temperature=0.1 which is close to 0 so mostly it will produce the same output results for tags and summary line but still not all the time it still may have some variations, so according to my understanding it can produce little different results for each model run.

Part 4

1. Create a reusable model-adaptor module at src/model_client.py, defining a stable interface such as complete(messages, tools=None). All model calls must go through this adaptor.

```

class ModelClient:
    """Adapter around a local Ollama chat model."""

    def __init__(self, model: str = MODEL_ID, temperature: float = 0.1) -> None:
        self._model_name = model
        self._llm = ChatOllama(model=model, temperature=temperature, reasoning=False)
        # Token usage (input/output/total) from the most recent complete() call, exposed for callers that want to report per-turn token counts.
        self.last_usage: Dict[str, int] = {"input_tokens": 0, "output_tokens": 0, "total_tokens": 0}

    def complete(
        self,
        messages: Sequence[Message],
        tools: Optional[List[Dict[str, Any]]] = None,
        *,
        response_format: Optional[str] = None,
        max_tokens: int = 4096,
    ) -> str:
        """Send `messages` to the model and return its raw text reply."""
        self._llm.num_predict = max_tokens
        self._llm.format = response_format
        model = self._llm.bind_tools(tools) if tools else self._llm

        try:
            response = model.invoke(_to_langchain_messages(messages))
        except Exception as exc: # connection refused, model not pulled, daemon down
            raise ModelClientError(
                f"Could not reach Ollama model '{self._model_name}'. Make sure `ollama serve` "
                f"is running and `ollama pull {self._model_name}` has completed. ({exc})"
            ) from exc

        usage = response.usage_metadata or {}
        self.last_usage = {key: usage.get(key, 0) for key in self.last_usage}
        return response.content

    def call(
        self,
        *,
        system: str,
        user: str,
        output_format: Type[T],
        max_tokens: int = 4096,
        max_attempts: int = DEFAULT_MAX_ATTEMPTS,
    ) -> T:
        """Call the model and parse+validate its JSON reply as `output_format`."""
        messages: List[Message] = [
            {"role": "system", "content": system},
            {"role": "user", "content": user},
        ]

        last_error: Optional[Exception] = None
        last_raw = ""

        for _attempt in range(1, max_attempts + 1):
            last_raw = self.complete(messages, response_format="json", max_tokens=max_tokens)

```

2. Create `hw1_client.py` as a small command-line demo that imports the adapter.

code >  hw1_client.py >  main

```
51
52
53 def print_exit_summary(stats: ConversationStats) -> None:
54     print("--- session end ---")
55     print(f"cumulative input tokens: {stats.input_tokens}")
56     print(f"cumulative output tokens: {stats.output_tokens}")
57     print(f"turn count: {stats.turns}")
58
59
60 def main() -> None:
61     client = ModelClient()
62     history: List[Dict[str, str]] = [{"role": "system", "content": load_system_prompt()}]
63     stats = ConversationStats()
64
65     print("hw1_client.py - type a message, /stats, or /exit (Ctrl-D also exits).")
66
67     while True:
68         try:
69             user_input = input("> ").strip()
70         except EOFError:
71             print()
72             break
73
74         if not user_input:
75             continue
76         if user_input == "/exit":
77             break
78         if user_input == "/stats":
79             print_stats(stats, history)
80             continue
81
82         history.append({"role": "user", "content": user_input})
83         try:
84             reply = client.complete(history)
85         except ModelClientError as exc:
86             print(f"Model call failed: {exc}", file=sys.stderr)
87             history.pop()
88             continue
89
90         history.append({"role": "assistant", "content": reply})
91         stats.record(client.last_usage)
92
93         print(reply)
94         print_turn_usage(client.last_usage)
95
96     print_exit_summary(stats)
97
```

3. Create AGENT.md asking for strict bullet-only code review, and verify the model actually follows it.

```
AGENT.md > # AGENT.md > ## Rules
1 # AGENT.md
2
3 System prompt for [hw1_client.py](hw1_client.py), loaded verbatim as the system message for
4 every turn of its conversation. Defines a strict, bullet-only code-review persona, used to
5 test whether the model actually honors a formatting constraint under real content pressure
6 across a multi-turn conversation (not just the first reply).
7
8 ## Rules
9
10 You are a strict code reviewer. Follow these rules with no exceptions, on every single response:
11
12 - Respond only with a bulleted list. Every line must start with '- '.
13 - Never write introductory or concluding prose (no "Here's my review:", no "Overall, ...", no summaries).
14 - Never use numbered lists, headings, tables, or code fences.
15 - Each bullet must be one specific, actionable review point (a bug, a risk, or a concrete improvement) – not a
    restatement of what the code does.
16 - If there is truly nothing to flag, respond with exactly one bullet: '- No issues found.'
17 - These rules apply to every turn in the conversation, including the fifth and beyond – do not relax formatting
    as the conversation continues.
```

4. After every model response, print input tokens, output tokens, and total tokens for that turn. On exit, print cumulative input tokens, output tokens, and turn count.

```
turn count: 3
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % python code/hw1_client.py
hw1_client.py - type a message, /stats, or /exit (Ctrl-D also exits).
> Review this code: def get_user(users, uid):\n    for u in users:\n        if u['id'] == uid:\n            return u

- The function does not handle the case where 'uid' is not found in 'users'.
[tokens] input=285 output=20 total=305
> > /stats
--- /stats ---
turns: 1
cumulative input tokens: 285
cumulative output tokens: 20
cumulative total tokens: 305
serialized conversation-history length: 1402 chars
-----
> /exit
--- session end ---
cumulative input tokens: 285
cumulative output tokens: 20
turn count: 1
```

5. Add a /stats command showing turn count, cumulative token counts, and serialized conversation-history length, without altering the history.

```
turn count: 1
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % python code/hw1_client.py
hw1_client.py - type a message, /stats, or /exit (Ctrl-D also exits).
> Review this function: def save_password(password):\n    with open('passwords.txt', 'a') as f:\n        f.write(password)

- The function appends the password to a file without adding a newline character, which may cause issues when reading the file later.
- The function does not handle exceptions that may occur during file operations.
- The function does not securely handle the password, as it is written directly to the file without encryption.
- The function does not validate the input to ensure it is a string.
- The function does not provide any feedback to the user about the success or failure of the operation.
[tokens] input=283 output=101 total=384
> > /stats
--- /stats ---
turns: 1
cumulative input tokens: 283
cumulative output tokens: 101
cumulative total tokens: 384
serialized conversation-history length: 1842 chars
-----
>
```

6. Run a five-turn conversation; record /stats output after turn 3 and after turn 5.

```

(.venv) (base) pc@pcs-MacBook-Pro data260-7117 % python code/hw1_client.py 2>61 | tee code/hw1_client_transcript.txt
hw1_client.py - type a message, /stats, or /exit (Ctrl-D also exits).
> def calculate_average(numbers):
    total = 0
    for n in numbers:
        total += n
    return total / len(numbers)

- Missing error handling for empty input list
- No type hints for function parameters
- No docstring explaining the function's purpose
- No validation for input type
- No return statement for empty list case
[tokens] input=258 output=43 total=301
> - Division by zero in the expression '0 /'
[tokens] input=314 output=12 total=326
> - Division by zero in the expression '0 /'
[tokens] input=340 output=12 total=352
> - Division by zero in the expression 'n /'
[tokens] input=364 output=12 total=376
> - Division by zero in the expression 'total / len(numbers)'
[tokens] input=391 output=14 total=405
> > def add_item(item, items=[]):
    items.append(item)
    return items

- Default argument mutation risk due to mutable list 'items=[]'
[tokens] input=422 output=14 total=436
> - Default argument mutation risk due to mutable list 'items=[]'
[tokens] input=449 output=14 total=463
> - Division by zero in the expression 'items /'
[tokens] input=474 output=12 total=486
> > Can you summarize your review so far in a short plain-English paragraph instead of bullets?

The code has several issues, including potential division by zero errors, missing error handling, lack of type hints, and improper use of mutable default arguments. These issues can lead to runtime errors and make the code less reliable and harder to understand.
[tokens] input=513 output=49 total=562
--- /stats ---
turns: 9
cumulative input tokens: 3525
cumulative output tokens: 182
cumulative total tokens: 3707
serialized conversation-history length: 2851 chars

> Can you summarize your review so far in a short plain-English paragraph instead of bullets?

The code has several issues, including potential division by zero errors, missing error handling, lack of type hints, and improper use of mutable default arguments. These issues can lead to runtime errors and make the code less reliable and harder to understand.
[tokens] input=589 output=49 total=638
> > def is_even(n):
    return n % 2 == 0

- Missing error handling for non-integer input
- No type hints for function parameters
- No docstring explaining the function's purpose
- No validation for input type
[tokens] input=652 output=35 total=687
> - Syntax error: Missing closing parenthesis for the modulo operation
- Division by zero in the expression '0 /'
[tokens] input=704 output=24 total=728

--- /stats ---
turns: 14
cumulative input tokens: 6968
cumulative output tokens: 314
cumulative total tokens: 7282
serialized conversation-history length: 3972 chars

> /exit
--- session end ---
cumulative input tokens: 6968
cumulative output tokens: 314
turn count: 14
(.venv) (base) pc@pcs-MacBook-Pro data260-7117 %

```

```
hw1_client_transcript.txt U x ...
code > hw1_client_transcript.txt
1 hw1_client.py - type a message, /stats, or /exit (Ctrl-D also exits).
2 > - Missing error handling for empty input list
3 - No type hints for function parameters
4 - No docstring explaining the function's purpose
5 - No validation for input type
6 - No return statement for empty list case
7 [tokens] input=258 output=43 total=301
8 > - Division by zero in the expression '0 /'
9 [tokens] input=314 output=12 total=326
10 > - Division by zero in the expression '0 /'
11 [tokens] input=340 output=12 total=352
12 > - Division by zero in the expression 'n /'
13 [tokens] input=364 output=12 total=376
14 > - Division by zero in the expression 'total / len(numbers)'
15 [tokens] input=391 output=14 total=405
16 >> - Default argument mutation risk due to mutable list 'items=[]'
17 [tokens] input=422 output=14 total=436
18 >> - Default argument mutation risk due to mutable list 'items=[]'
19 [tokens] input=449 output=14 total=463
20 > - Division by zero in the expression 'items /'
21 [tokens] input=474 output=12 total=486
22 >> The code has several issues, including potential division by zero errors, missing error handling, lack of type hints, and improper use of mutable default
23 [tokens] input=513 output=49 total=562
24 >> --- /stats ---
25 turns: 9
26 cumulative input tokens: 3525
27 cumulative output tokens: 182
28 cumulative total tokens: 3707
29 serialized conversation-history length: 2851 chars
30 -----
31 > The code has several issues, including potential division by zero errors, missing error handling, lack of type hints, and improper use of mutable default
32 [tokens] input=589 output=49 total=638
33 >> - Missing error handling for non-integer input
34 - No type hints for function parameters
35 - No docstring explaining the function's purpose
36 - No validation for input type
37 [tokens] input=652 output=35 total=687
38 > - Syntax error: Missing closing parenthesis for the modulo operation
39 - Division by zero in the expression '0 /'
40 [tokens] input=704 output=24 total=728
41 >> - Division by zero in the expression 'stats /'
42 [tokens] input=738 output=12 total=750
43 > - Division by zero in the expression 'exit /'
44 [tokens] input=760 output=12 total=772
45 > --- /stats ---
46 turns: 14
47 cumulative input tokens: 6968
48 cumulative output tokens: 314
49 cumulative total tokens: 7282
50 serialized conversation-history length: 3972 chars
51 -----
52 > --- session end ---
53 cumulative input tokens: 6968
54 cumulative output tokens: 314
55 turn count: 14
56
```

7. In your Report and README, answer: why is prior conversation context resent with every turn? How is a system prompt different from a user message? Why do input tokens grow over a conversation? What eventually limits that growth?

Readme file-

history[0]) sets standing instructions and persona for the entire conversation here, "respond only as a strict bullet-only code reviewer." It's sent once and never changes. User messages are the turn-by-turn content the conversation is actually about (the code snippets being reviewed). Each one is new, appended once, and doesn't restate the rules. Models are trained to give system-role instructions priority over user-role content, so the system prompt is the mechanism for asking "how should you behave for every reply," while user messages are "what should you respond to right now." The turn 3/5 pressure tests in the transcript, which ask the model to drop the bullet format, exercise exactly this: whether a user message can override a standing system instruction.

Why do input tokens grow over a conversation?

Because history only ever grows each turn appends a user message and, after a successful reply, an assistant message, in the main() loop of hw1_client.py and the entire history is resent on every call. Turn N's input therefore contains the system prompt plus all 2*(N-1) prior messages plus the new one. The recorded transcript shows this directly: per-turn input tokens climbed monotonically as the conversation went on 267 on turn 1, 314 on turn 2, 381 on turn 3, 441 on turn 4, and 502 on turn 5. The /stats "serialized conversation-history length" (len(json.dumps(history))), which went from 1888 characters after turn 3 to 2451 characters after turn 5, grows for the same reason it's the same accumulating list measured a different way. Output tokens, by contrast, stay roughly flat per turn, around 16 to 24 tokens, since each reply only has to cover that turn's content.

What eventually limits that growth?

The model's context window is the fixed maximum number of tokens, input plus output combined, that qwen3:8b can process in a single call. Since input size grows roughly linearly with turn count while the window is fixed, a long enough conversation will eventually exceed it. hw1_client.py has no mitigation for this no truncation, sliding window, or summarization of older turns, so it will keep resending the full, ever-growing history until a call fails or is truncated by the model backend. In a production client, this is normally handled by trimming the oldest turns, summarizing older history into a shorter form, or capping total conversation length before that limit is hit.